The Random-Oracle Model, RSA-FDH, and Schnorr Signatures

CS 171: Cryptography

Sanjam Garg

October 27, 2026

Overview

1. Signatures: Recall
2. The Random-Oracle Model
3. RSA-FDH Signatures
4. Schnorr Signatures
5. Summary

Signatures: Recall

Signature scheme $\Pi = (\mathsf{Gen}, \mathsf{Sign}, \mathsf{Vrfy})$

$\mathsf{Gen}(1^n) \to (pk, sk)$; $\mathsf{Sign}_{sk}(m) \to \sigma$; $\mathsf{Vrfy}_{pk}(m, \sigma) \to 0/1$. Correctness: $\mathsf{Vrfy}_{pk}(m, \mathsf{Sign}_{sk}(m)) = 1$.

Game $\mathsf{SigForge}_{\mathcal{A},\Pi}(n)$

$\mathcal{A}$ gets 1^n, pk, and oracle access to $\mathsf{Sign}_{sk}(\cdot)$ (queried messages: M); it outputs (m^*, σ^*) and wins if $\mathsf{Vrfy}_{pk}(m^*, \sigma^*) = 1$ and $m^* \notin M$. Secure: $\Pr[\mathsf{SigForge}_{\mathcal{A},\Pi}(n) = 1] \leq \mathsf{negl}(n)$.

Last Lecture, and Today

Last lecture

- **Textbook RSA** signatures $\sigma = m^d \bmod N$ are insecure: no-message and multiplicative forgeries.
- **Hash-and-sign**: sign $H_s(m)$ — from short messages to any length.
- The **Schnorr identification scheme**, secure under DLog (by rewinding).

Today

Two signature schemes used in practice — **RSA-FDH** and **Schnorr** — whose proofs need a new tool: the **random-oracle model**.

Can a Hash Function Look Random?

- We often use a hash function **as if its outputs were random**:
  - the ElGamal KEM derives its key as $k := H(h^r)$ (lecture 15);
- But what would "H is pseudorandom" even mean? H has **no secret**: anyone can compute $H(0^n)$ and compare.

Problem

A deterministic function without secret inputs can **never** look random to someone who can compute it. Collision resistance does not capture "random-looking" either.

Idea

Prove security in an **idealized model** in which H is a truly random function.

The Random-Oracle Model

Random oracle

$H : \{0,1\}^* \to \{0,1\}^\ell$ is a **uniformly random function**. **Everyone** — the scheme, the honest parties, and the adversary — can only **query** it, as an oracle.

Lazy sampling

A random function on an infinite domain cannot be written down, but it can be **sampled on the fly**:

- keep a table T, initially empty;
- on query x: if $T[x]$ is undefined, set $T[x] \leftarrow \{0,1\}^\ell$ uniformly; return $T[x]$.

- Security games are the same as before, except that all parties get oracle access to H, and probabilities are also over the choice of H.
- In practice, H is then **instantiated** with a concrete hash function such as SHA-256.

What a Proof Can Do in the Random-Oracle Model

A reduction $\mathcal{B}$ that runs $\mathcal{A}$ also answers $\mathcal{A}$'s queries to H. This gives it three powers:

1. **Uniformity**: if x has not been queried, $H(x)$ is uniform — even given all other answers.
2. **Observing**: $\mathcal{B}$ sees **every** query that $\mathcal{A}$ makes to H.
3. **Programming**: $\mathcal{B}$ may **choose** the answers, as long as they are distributed uniformly (and consistently: the same x always gets the same answer).

Note

None of this is possible for a fixed, public function like SHA-256: anyone can evaluate it without asking anybody.

Example: A PRF from a Random Oracle

Construction

$F_k(x) := H(k \| x)$, with $k \leftarrow \{0,1\}^n$. Claim: F is a PRF in the random-oracle model.

- $\mathcal{A}$ queries F_k and, now, also H. Each $F_k(x)$ is H at the point $k \| x$.
- **Uniformity**: unless $\mathcal{A}$ itself queries H on some $k \| x'$, the values $F_k(x)$ are fresh uniform values — exactly a random function.
- **Observing**: querying H on $k \| x'$ means guessing k. With q queries, this happens with probability at most $q/2^n$: negligible.

Is the Random-Oracle Model Sound?

- No real hash function is a random oracle: SHA-256 has a short, public description.
- There are (contrived) schemes that are secure in the random-oracle model but **insecure** for **every** concrete choice of H [Canetti–Goldreich–Halevi 1998].
- Still, a random-oracle proof rules out every attack that treats H as a black box, and the schemes proved this way — including today's two — have held up well in practice.

Takeaway

"All models are wrong, but some are useful" (George Box). A random-oracle proof is strong evidence, **not** a proof under a standard assumption alone.

RSA-FDH (Full-Domain Hash)

Construction

Let $H : \{0,1\}^* \to \mathbb{Z}_N^*$. $\mathrm{Sign}_{sk}(m)$: $\sigma := H(m)^d \bmod N$. $\mathrm{Vrfy}_{pk}(m, \sigma)$: 1 iff $\sigma^e = H(m) \bmod N$.

- Last lecture's **no-message forgery?** Picking σ first gives σ^e; a forgery needs an m with $H(m) = \sigma^e$ — inverting H.
- The **multiplicative forgery?** $H(m_1) \cdot H(m_2)$ has nothing to do with $H(m_1 m_2)$.
- Like hash-and-sign, it signs messages of any length. But collision resistance of H is **not** known to suffice: the proof models H as a random oracle onto all of $\mathbb{Z}_N^*$ ("full domain").

Theorem 1

If the RSA problem is hard relative to GenRSA *and H is modeled as a random oracle, then RSA-FDH is existentially unforgeable.*

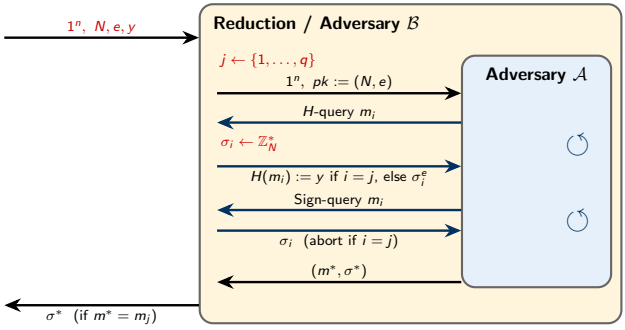
RSA-FDH: Proof Idea

Simplifying assumptions (without loss of generality)

$\mathcal{A}$ makes $q(n)$ distinct queries $m_1, m_2, \ldots$ to H, and queries $H(m)$ before asking for $\mathsf{Sign}_{sk}(m)$ and before outputting (m, σ^*).

- $\mathcal{B}$ gets (N, e, y) and must find x with $x^e = y$. It has **no** d — so how does it answer $\mathcal{A}$'s signing queries?
- **Programming**: on the query m_i, $\mathcal{B}$ samples $\sigma_i \leftarrow \mathbb{Z}_N^*$ and sets $H(m_i) := \sigma_i^e$. This answer is uniform ($x \mapsto x^e$ is a permutation of $\mathbb{Z}_N^*$), and $\mathcal{B}$ knows the signature σ_i on m_i.
- But then a forgery teaches $\mathcal{B}$ nothing: it already knows every signature. **Embedding**: $\mathcal{B}$ guesses $j \leftarrow \{1, \ldots, q\}$ and sets $H(m_j) := y$ instead. A forgery on m_j is a σ^* with $(\sigma^*)^e = y$!

RSA-FDH: Proof by Reduction



Given / To prove

Given: $\Pr\left[\mathrm{SigForge}_{\mathcal{A},\Pi}(n) = 1\right] \geq \epsilon(n)$ for non-negligible ϵ, with $q(n)$ queries to H.
To prove: $\Pr\left[\mathrm{RSA}_{\mathcal{B},\mathrm{GenRSA}}(n) = 1\right] \geq \epsilon(n)/q(n)$.

RSA-FDH: Putting It Together

- **$\mathcal{A}$'s view is as in** SigForge: every H-answer is uniform in $\mathbb{Z}_N^*$ (y is uniform, and so is σ_i^e), and every signature is valid: $\sigma_i^e = H(m_i)$.
- **j is hidden**: the answers are distributed the same way whatever j is. So if $\mathcal{A}$ forges on $m^* = m_k$, then $j = k$ with probability $1/q$.
- **No abort when it matters**: if $j = k$, then $m_j = m^*$ was never signed, so $\mathcal{B}$ never had to sign m_j; and $(\sigma^*)^e = H(m_j) = y$.

Success probability of $\mathcal{B}$

$$\Pr\left[\mathrm{RSA}_{\mathcal{B},\mathrm{GenRSA}}(n) = 1\right] \;\geq\; \Pr\left[\mathrm{SigForge}_{\mathcal{A},\Pi}(n) = 1\right] \cdot \frac{1}{q(n)} \;\geq\; \frac{\epsilon(n)}{q(n)},$$

non-negligible — contradicting the RSA assumption. □

Remark (in practice)

$\mathcal{B}$ is PPT, but loses a factor q. The standard **RSA-PSS** signs $H(m \,\|\, r)^d$ with a fresh random r, which allows a proof without this loss (JoC 16.2).

Identification Schemes: Recall

Identification scheme $(\mathsf{Gen}, P_1, P_2, V)$

Prover: $(I, st) \leftarrow P_1(sk)$, sends I.
Verifier: sends a challenge $r \leftarrow \Omega_{pk}$.
Prover: sends $s := P_2(sk, st, r)$.
Verifier: accepts iff $V(pk, r, s) = I$.

Schnorr

$pk = (G, q, g, h = g^x)$, $sk = x$.
$I := g^k$ for $k \leftarrow \mathbb{Z}_q$; $r \leftarrow \mathbb{Z}_q$;
$s := rx + k \bmod q$; $V(pk, r, s) = g^s / h^r$.

Passive security $(\mathsf{Ident}_{\mathcal{A},\Pi}(n))$

Given pk and honest transcripts (I, r, s) from Trans_{sk}, $\mathcal{A}$ sends I^*, receives a uniform r^*, and must answer with s^* such that $V(pk, r^*, s^*) = I^*$.

- Last lecture: Schnorr identification is passively secure if DLog is hard (by **rewinding**).

From Identification Scheme to Signatures

The Fiat–Shamir transform

$\mathsf{Gen}(1^n)$

$(pk, sk) \leftarrow \mathsf{Gen}_{id}(1^n)$, and a hash function $H : \{0,1\}^* \to \Omega_{pk}$.

$\mathsf{Sign}_{sk}(m)$

$(I, st) \leftarrow P_1(sk)$; $r := H(I, m)$; $s := P_2(sk, st, r)$; output $\sigma = (I, s)$.

$\mathsf{Vrfy}_{pk}(m, \sigma)$

Output 1 iff $V\big(pk, H(I, m), s\big) = I$.

Remark

The verifier's random challenge is replaced by a hash of I and m. A forger cannot pick I after seeing the challenge, because the challenge depends on I — **if** $H(I, m)$ behaves like a fresh random value: a random oracle.

Schnorr Signatures

$\mathsf{Gen}(1^n)$

$(G, q, g) \leftarrow \mathcal{G}(1^n)$; $x \leftarrow \mathbb{Z}_q$, $h := g^x$; $H : \{0,1\}^* \to \mathbb{Z}_q$. $pk = (G, q, g, h)$, $sk = x$.

$\mathsf{Sign}_{sk}(m)$

$k \leftarrow \mathbb{Z}_q$, $I := g^k$;
$r := H(I, m)$;
$s := rx + k \bmod q$. Output $\sigma = (I, s)$.

$\mathsf{Vrfy}_{pk}(m, (I, s))$

$r := H(I, m)$;
output 1 iff $g^s / h^r = I$.

- **Correctness**: $g^s / h^r = g^{rx+k} / g^{xr} = g^k = I$, exactly as for the identification scheme.
- Short and fast (with elliptic-curve groups), and widely deployed, e.g. in variants such as EdDSA.

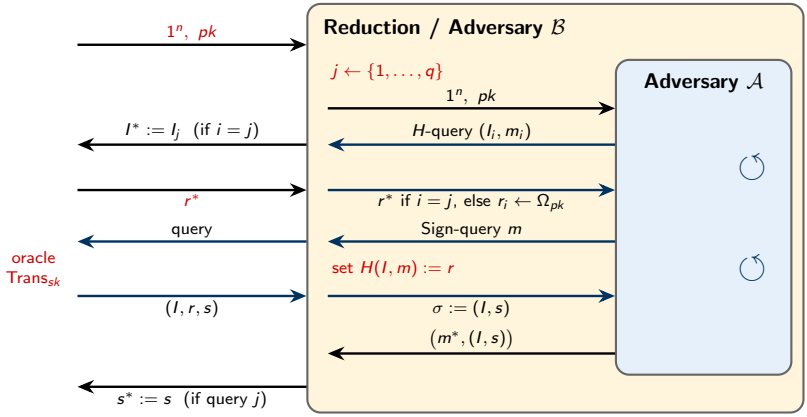
Fiat–Shamir: Security

Theorem 2

If the identification scheme is passively secure, I has high entropy, and H is modeled as a random oracle, then the Fiat–Shamir signature scheme is existentially unforgeable.

- "High entropy": any fixed value equals I with negligible probability. For Schnorr, $I = g^k$ is uniform in G.
- **Proof idea.** From a forger $\mathcal{A}$, build $\mathcal{B}$ that attacks the identification scheme, answering $\mathcal{A}$'s queries to H and to Sign_{sk} itself:
  - **Signing without** sk by **programming** H: take an honest transcript (I, r, s) from Trans_{sk}, and declare $H(I, m) := r$.
  - **Embedding the challenge** (as for RSA-FDH): one of $\mathcal{A}$'s queries $H(I_j, m_j)$ is answered with the verifier's challenge r^*. A forgery $(m_j, (I_j, s))$ then contains an answer s to that challenge.

Fiat–Shamir: Proof by Reduction



Fiat–Shamir: Putting It Together

Given / To prove ($q(n)$ distinct H-queries, including (I, m^*))

Given: $\Pr\left[\mathsf{SigForge}_{\mathcal{A},\Pi}(n) = 1\right] \geq \epsilon(n)$ for non-negligible ϵ.
To prove: $\Pr\left[\mathsf{Ident}_{\mathcal{B},\Pi_{id}}(n) = 1\right] \geq \epsilon(n)/q(n) - \mathsf{negl}(n)$.

- $\mathcal{A}$**'s view is as in** SigForge: all H-answers are uniform, and each (I, s) is valid: (I, r, s) is an accepting transcript with $r = H(I, m)$. Programming fails only if $H(I, m)$ is already defined: negligible, as I has high entropy.
- j **is hidden**, so a forgery uses the j-th query with probability $1/q$. That query was not programmed by a Sign-query, since m^* was never signed. Then $V(pk, r^*, s) = I_j$: $\mathcal{B}$ wins.

Success probability of $\mathcal{B}$

$\Pr\left[\mathsf{Ident}_{\mathcal{B},\Pi_{id}}(n) = 1\right] \geq \epsilon(n)/q(n) - \mathsf{negl}(n)$: non-negligible — contradicting passive security. □

Schnorr Signatures: Putting the Pieces Together



Corollary

If DLog is hard relative to $\mathcal{G}$ and H is modeled as a random oracle, then Schnorr signatures are existentially unforgeable.

The pattern, again

As for RSA-FDH: **program** H to answer signing queries without sk, and **embed** the challenge at a guessed query. Both are impossible for a fixed, public H.

Summary

- **Random-oracle model**: H is a uniformly random function that everyone can only query. Proofs may rely on **uniformity**, **observe** $\mathcal{A}$'s queries, and **program** the answers. A heuristic: no real hash function is a random oracle.
- **Example**: $F_k(x) = H(k \,\|\, x)$ is a PRF in the random-oracle model (unless $\mathcal{A}$ queries H on $k \,\|\, x'$, i.e. guesses k).
- **RSA-FDH** $\sigma = H(m)^d$ is secure under RSA. *Proved*: program $H(m_i) := \sigma_i^e$, embed y at a guessed query (loss $1/q$). In practice: RSA-PSS.
- **Fiat–Shamir**: $r := H(I, m)$ turns a passively secure identification scheme into a signature scheme. *Proved*: the same pattern — sign by programming H with honest transcripts, embed the challenge at a guessed query. **Schnorr signatures** are secure if DLog is hard.